

ESOF 322

Software Engineering

LECTURE: 11

J. L. Pach

OUTLINE PR

- System Modeling

The One Idea to Remember

> A model is a **deliberately simplified view** of a system, built to **answer a specific question**.

- No single model shows everything. You need **several views**.
- A model is useful when it **helps people communicate or decide**.
- In agile work, the question is not "Is the model complete?" but:
  "Is it good enough for what we need right now?"

After This Chapter You Should Be Able To

1. Explain **why** teams model and **when** modeling is worth the time
2. Choose the **right diagram** for a given question
3. Draw and read **context, use case, sequence, class, and state** diagrams
4. Explain how **agile modeling** differs from documentation-heavy modeling
5. Explain what **model-driven engineering** is and why agile teams are skeptical of it

Three Uses of Models (Know the Difference)

Use	Must be complete?	Must be correct?	Agile fit
Discussion (whiteboard, sketch)	No	Roughly	★★★ Core practice
Documentation of existing system	No	**Yes**	★★ Keep it small
Code generation (MDE)	**Yes**	**Yes**	★ Rare

Understand: Most agile modeling is the first kind. It is fine to throw a sketch away after it has done its job.

Agile Modeling: The Mindset

- **Model with a purpose**: know what question the diagram answers
- **Just barely good enough**: stop when the team understands
- **Model in small increments**: sketch for the current user story, not the whole system
- **Prefer simple tools**: whiteboard, paper, Mermaid in the repo
- **Keep only the "keepers"**: update a model only if someone will read it again
- **Working software over comprehensive documentation** (Agile Manifesto)

Know: Agile does **not** mean "no models". It means **no models without a reason**.

Four Perspectives

Perspective	Question it answers	Typical diagram
Context	What is inside and outside our system?	Context diagram
Interaction	Who talks to what, and in what order?	Use case, sequence
Structure	What are the parts and how do they relate?	Class diagram
Behavior	How does the system react over time?	Activity, state

Be able to: given a question from a teammate, name the perspective and the diagram.

UML: Five Diagrams Are Enough

UML has many diagram types (Sommerville cites 13). Research (Erickson & Siau, 2007) shows most practitioners use **five**:

1. **Activity** – steps in a process
2. **Use case** – interactions with the environment
3. **Sequence** – order of messages between participants
4. **Class** – structure and relationships
5. **State** – reactions to events

Know these five well. Recognize the others exist.

CONTEXT MODELS

Context: Drawing the System Boundary

Understand:

- The boundary decides **what we build** and **what we depend on**
- It is a **business and political decision**, not only a technical one
- Deciding early saves effort, and in agile it is **revisited** as we learn

Agile link: the boundary defines the **scope of the product backlog**. Items outside the boundary are dependencies, not stories.

Context Example: Our Miner Game

```
flowchart LR
    Player((Player)) -->|keyboard| Game[Miner Game]
    Game -->|window, drawing, input| Raylib[raylib]
    Game -->|loads| Assets[(PNG assets)]
    Raylib --> OS[Windows / OpenGL]
```

Limitation to remember: a context diagram shows **what** is around the system, not **how** data flows. Use a process or activity model for that.

Activity Diagrams

Read and draw:

- filled circle = start, circle-in-circle = end
- rounded box = activity, arrow = flow
- diamond = decision, thick bar = fork/join (parallel work)
- swimlanes = **who** does each step

Agile use: sketching a **workflow** with the Product Owner before writing stories, for example "what happens when a pull request is opened".

INTERACTION MODELS

Use Cases vs. User Stories

	Use case	User story
Origin	Jacobson, 1990s (UML)	XP / Scrum
Form	Diagram + structured description	One sentence + acceptance criteria
Scope	One complete interaction	A small, valuable slice
Goal	Specify behavior	**Start a conversation**

> As a player, I want to dig through earth, so that I can reach ore.

Understand: a use case can be split into **several user stories**. Agile teams usually prefer stories, and use cases help when interactions are complex.

Use Case Diagram

```
flowchart LR
    P((Player)) --- U1([Move miner])
    P --- U2([Dig earth])
    P --- U3([Collect ore])
    P --- U4([Eat food])
```

Know:

- actors (people or other systems) are **outside** the system
- one use case = **one recognizable goal** of an actor
- the diagram alone is weak; the value is in the **scenario text**

Sequence Diagrams

Read and draw:

- participants at the top, **lifelines** going down
- time flows **top to bottom**
- arrows = messages or calls; dashed arrows = returns
- `alt` / `opt` / `loop` boxes = alternatives, optional steps, repetition

Agile use: a quick sketch **before coding a story** that touches several modules. It reveals who calls whom and where the responsibilities belong.

Sequence Example: Digging

```
sequenceDiagram
    participant I as Input
    participant P as Physics
    participant M as Map
    participant R as Render
    I->>P: dig pressed (direction)
    P->>M: digCell(x, y)
    alt cell is EARTH
        M-->>P: cell cleared
        P->>P: stamina -= 15
    else cell is EMPTY
        M-->>P: nothing
    end
    R->>M: draw grid
```

> **Notice:** this diagram assumes the **MVC split we have not done yet**. Models can describe the **target** design, too.

STRUCTURAL MODELS

Class Diagrams

Know the notation:

- box = class (name, attributes, operations)
- line = **association**, with multiplicity: `1`, `0..1`, `*`, `1..*`
- hollow triangle = **generalization** (inheritance)
- diamond = **aggregation** (whole–part)

Understand the two uses:

- **Analysis:** real-world concepts (Player, Ore, Map), no implementation details
- **Design:** actual classes with types and methods

Class Example: Target Design

```
classDiagram
    class GameState {
        +int stamina
        +int score
        +update()
    }
    class Map {
        +Cell cells[ROWS][COLS]
        +digCell(x, y)
        +isSolid(x, y)
    }
    class Miner {
        +Vector2 position
        +AnimState state
    }
    GameState "1" *-- "1" Map
    GameState "1" *-- "1" Miner
```

Agile tip: for existing code, **generate or sketch class diagrams from code**. Do not maintain a large one by hand.

Generalization and Aggregation

Generalization

- move common attributes and operations to a **superclass**
- subclasses add or override only what is specific
- example: `Tile` → `EarthTile`, `OreTile`, `FoodTile`

Aggregation / composition

- "is made of": `Map` is made of `Cell`s

Understand: these ideas lead directly to **design patterns** later in the course (Strategy, Decorator).

BEHAVIORAL MODELS

Data-Driven vs. Event-Driven

	Data-driven	Event-driven
Triggered by	arriving data	events (input, timers)
Typical systems	business, batch processing	games, GUIs, embedded
Typical diagram	activity / data flow	**state diagram**

Be able to: classify a system and choose the model. Our game is **event-driven**.

State Diagrams

Know:

- an object is always in **exactly one state** at a time
- **events** (key pressed, timer, collision) cause **transitions**
- rounded box = state, arrow = transition labeled with the event
- filled circle = initial state, circle-in-circle = final state
- optional **guard** in brackets: `jump [on ground]`

Understand: a state diagram answers *"what can happen next, and what cannot?"* Missing transitions are often **bugs waiting to happen**.

State Example: The Miner

```
stateDiagram-v2
    [*] --> Idle
    Idle --> Walking : left / right pressed
    Walking --> Idle : key released
    Idle --> Jumping : jump [on ground]
    Walking --> Jumping : jump [on ground]
    Jumping --> Falling : velocity > 0
    Falling --> Idle : landed
    Idle --> Digging : dig pressed
    Digging --> Idle : cell cleared
    Idle --> [*] : stamina == 0
```

Look at our prototype: every state should have its **own animation**. The code reuses the idle sprite for walking. The diagram makes the gap obvious, while the code hides it.

State Diagrams in Agile Work

- Draw one **before** implementing a story that adds a new state (for example "As a player, I want to climb ladders")
- Use it to write **acceptance criteria**: one criterion per important transition
- Use it to write **unit tests**: *given state X, when event E, then state Y*
- Keep it in the repo as **Mermaid**, so it is versioned and reviewed in pull requests like code

Be able to: turn a state diagram into a list of test cases.

What Is MDE?

- **Models, not programs**, are the main output of development
- Code is **generated automatically** from models
- Goal: work at a higher level of abstraction, independent of platform

MDA (Model-Driven Architecture, OMG) uses three levels:

Model	Meaning
CIM – Computation Independent	the domain, no software details
PIM – Platform Independent	system design without technology
PSM – Platform Specific	design for a concrete platform (Java, .NET, ...)

Transformations: `CIM → PIM → PSM → code`, ideally automated.

MDE and Agile: Know the Tension

Why agile teams are skeptical:

- MDE needs **extensive, complete models up front**, which conflicts with incremental delivery
- requires **specialized tools** and skills
- models good for discussion are often **not detailed enough** to generate code
- platform independence matters less when the team picked one platform

Sommerville's nuance: if transformation is **fully automated** (for example *Executable UML*), the model *becomes* the program, and agile practices can apply to the model itself.

Where MDE works well: embedded and safety-critical systems, protocol and code generators, database schemas.

You Already Use a Little MDE

Small, practical examples of "model → generated artifact":

- ⚒ **Mermaid text → rendered diagram** on GitHub
- ⚒ **Database migration files → schema** (Flyway, Liquibase)
- ⚒ **OpenAPI spec → client and server stubs**
- ⚒ **Doxygen comments → documentation site**

Understand: agile teams accept generation when the model is **small, versioned, and cheap to change**.

PUTTING IT TOGETHER

Which Diagram Should I Draw?

Question from a teammate	Draw
"What do we depend on? What is out of scope?"	**Context**
"What steps happen, and who does them?"	**Activity**
"What can the user do with the system?"	**Use case** (or user stories)
"Who calls whom when this story runs?"	**Sequence**
"What are the parts and how are they related?"	**Class**
"What can happen next from this situation?"	**State**

Common Mistakes

- Drawing a diagram **without a question** it should answer
- Trying to model **the whole system** at once
- Keeping diagrams that **no longer match the code**, which is worse than having none
- Mixing **analysis** (concepts) and **design** (implementation) in one class diagram
- Treating UML notation as decoration: wrong arrowheads **change the meaning**
- Using a diagram instead of **talking to the team**

Self-Check Questions

1. Why is no single model enough to describe a system?
2. Which of the three uses of models needs complete and correct models? Why?
3. How is a use case different from a user story? When would you use each?
4. What does a context diagram **not** show?
5. Name two things a sequence diagram reveals that a class diagram does not.
6. Is our game data-driven or event-driven? Which diagram follows from that?
7. How can a state diagram help you write unit tests?
8. Why do agile teams rarely use full MDA?

Apply It to the Miner Project

Add a `docs/models.md` file to your fork with **Mermaid** diagrams:

1. **Context diagram** of the game and its dependencies
2. **State diagram** of the miner, based on what the code *actually* does
3. **Sequence diagram** for one action (dig, eat, or collect ore)
4. **Class diagram** of the *target* structure after the MVC split

For each diagram, write **one sentence**: *what question does it answer?*

Then compare diagram 2 with the `notes.txt` known issues. Which bugs can you see in the model?

Key Takeaways

- A model is a **simplified view built for a purpose**
- Use **several perspectives**: context, interaction, structure, behavior
- Five UML diagrams cover most needs
- Agile modeling = **just enough, just in time**, mostly for communication
- Keep useful models **in the repo as text** (Mermaid), reviewed like code
- MDE is powerful in specific domains, but it conflicts with typical agile workflows

THANK

You